2021wb15553-berad Prashant Sopan .Berad

Prashant_Sopan_Berad_2021wb1553_MidSem_Report.docx

S2-25_SSWTZG628T - Mid Sem

S2-25_SSWTZG628T

Birla Institute of Technology and Science Pilani WILP Division

Document Details

Submission ID
trn:oid:::1:3558917234

Submission Date
May 5, 2026, 1:53 AM GMT+5:30

Download Date
May 5, 2026, 2:00 AM GMT+5:30

File Name
Prashant_Sopan_Berad_2021wb1553_MidSem_Report.docx

File Size
1.8 MB

28 Pages

3,410 Words

20,173 Characters

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

22 Not Cited or Quoted 10%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
1% Publications
3% Submitted works (Student Papers)

Match Groups

- **22** Not Cited or Quoted 10%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
1% Publications
3% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.coursehero.com 6%
2	Internet	www.slideshare.net 1%
3	Student papers	University of Hertfordshire <1%
4	Internet	desklib.com <1%
5	Student papers	Vellore Institute of Technology <1%
6	Internet	www.termpaperwarehouse.com <1%
7	Internet	pdffox.com <1%
8	Internet	indah.ump.edu.my <1%
9	Internet	vdocuments.site <1%
10	Internet	cs.stanford.edu <1%

11 Internet

etd.aau.edu.et <1%

12 Internet

repository.riarauniversity.ac.ke:8080 <1%

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

S2-25_SSWTZG628T DISSERTATION

CORPORATE CARPOOLING SYSTEM

A Geo-Aware Ride-Sharing Application for Same-Employer Commuters Featuring Live Tracking, Failover Driver Assignment, and an Emergency SOS Channel

MID-SEMESTER REPORT

Submitted in partial fulfilment of the requirements for the

M.Tech Software Engineering Degree Program

by

Prashant Sopan Berad

BITS ID: 2021wb15553

Under the guidance of

Prasanna Tukaram Desai

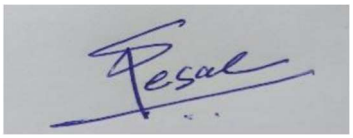
[Team Lead, Wipro Technologies Ltd., Pune]

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CERTIFICATE

It is hereby certified that the dissertation titled "Corporate Carpooling System", submitted by Prashant Sopan Berad (BITS ID No. 2021wb15553) toward partial fulfilment of the requirements of S2-25_SSWTZG628T Dissertation, represents the work that he has undertaken under my guidance.

The candidate has shown consistent advancement on the architecture and partial implementation of the platform — covering the database schema, the Spring Boot service tier, and the React-based front-end — consistent with the deliverables outlined in the original dissertation proposal.



Signature of the Supervisor

Name: Prasanna Tukaram Desai

Designation: Team Lead

ACKNOWLEDGEMENT

12 I wish to record my heartfelt gratitude to my dissertation supervisor, Mr. Prasanna Tukaram Desai, for his sustained guidance, technical insight, and unwavering patience over this first half of the project. The conversations we shared on architectural choices — particularly around leveraging PostGIS for route matching and the safety expectations placed on a corporate ride-sharing product — have meaningfully shaped the direction of this work.

I also extend my thanks to my examiners, Mr. Palash Jain and Mrs. Dhanya K., for the time they invested and the constructive critique they provided during the review milestones. Their suggestion to prioritize a working core ride lifecycle ahead of optional capabilities was instrumental in helping me sequence the implementation in a realistic manner.

1 I am equally grateful to Birla Institute of Technology and Science, Pilani, and to the Work Integrated Learning Programmes Division, for hosting this dissertation course and offering the academic framework that made this work feasible.

Prashant Sopan Berad
2021wb15553

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

CSIWZG628T DISSERTATION

MID-SEMESTER EVALUATION FORM

Section I

(To be completed by the student and returned to the Supervisor)

Field	Details
ID No.	2021wb1553
Name of Student	Prashant Sopan Berad
Name of Supervisor	Prasanna Tukaram Desai
Name of the Examiner(s)	Palash Jain, Dhanya K.
Dissertation Title	Corporate Carpooling System

Section II

(To be completed by the Supervisor in consultation with the examiner(s))

Examiner / Supervisor remarks on the dissertation (mark Y or N)

1. Quantum of work

Item	Y/N
a. Justifiable as eight weeks of effort	Y
b. Work aligns with the commitments stated in the outline	Y

2. Type of work

Item	Y/N
a. Client assignment	N
b. Organization-specific task	N
c. General study or white paper	N
d. Other (academic / open-source product build)	Y

3. Nature of work

Item	Y/N
a. Routine in nature	N
b. Required creativity and analytical thinking	Y

If the answer above is "Y," please elaborate below.

The dissertation centres on engineering a geo-aware matching subsystem on top of PostgreSQL and PostGIS, a real-time tracking pipeline carried over Web Sockets, and an automated fail-over mechanism

for backup drivers. Each of these components demanded standalone design decisions and bespoke data modelling that go beyond routine CRUD development.

4. Evaluation methodology

Item	Y/N
a. Evaluation conducted via presentation to supervisor and examiner	Y
b. Evaluation conducted via Viva by supervisor and examiner	Y
c. Student engaged regularly with the supervisor and acted on suggestions	Y

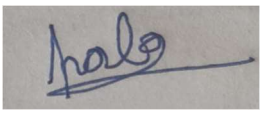
d. Brief summary of the report submitted, presentation quality, and improvement suggestions.

The mid-semester report follows the BITS dissertation template and walks through the introduction, literature survey, system analysis, system design with several UML diagrams, the implementation status of the back-end and front-end, and a detailed plan for the work that remains. The accompanying presentation has been kept brief and is reinforced by the architectural and lifecycle diagrams embedded within this report.

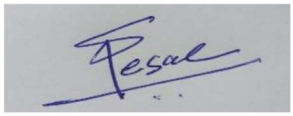
5. Mid-semester evaluation matrix

Dimension	1	2	3	4	5
Understanding of the dissertation subject					✓
Creative thinking and ability to generate new ideas				✓	
Communication ability				✓	
Organization of material					✓
Response to review questions				✓	
Cohesive thinking ability				✓	
Report structure and format					✓
Technical content of the report				✓	
Explanation of the assignment's significance				✓	
Analysis of alternative approaches				✓	

Date: 05-05-2026



Signature of Examiner(s)



Signature of Supervisor

TABLE OF CONTENTS

ABSTRACT

The everyday commute into metropolitan business districts is overwhelmingly carried out in privately owned cars carrying a single occupant. The visible consequences are jammed approach roads outside corporate parks, sprawling parking infrastructure that is expensive to build and operate, and a steady climb in personal travel expenses for the average employee. Public ride-hailing platforms address part of the symptom, but they accept the public at large, ignore the boundary of the employer, and impose a recurring fare that the company seldom underwrites.

This dissertation presents a Corporate Carpooling System that is open exclusively to verified employees of a participating organization. Drivers from inside the company list recurring or ad-hoc rides together with the planned route and the available seats. Passengers raise pickup-and-drop requests, and a geo-aware matching engine — built on PostgreSQL with the PostGIS extension — surfaces the most relevant rides using proximity, route overlap, the desired time window, and remaining seat capacity. After a match is confirmed, the platform takes responsibility for live location streaming, in-ride messaging via Web Sockets, an SOS safety channel that notifies pre-registered guardians, and a backup driver workflow that triggers automatically the moment the primary driver is unreachable.

By the mid-semester checkpoint, the database schema, the core Spring Boot service tier covering sixteen domain entities and sixteen REST controllers, and a React single-page front-end with ten user-facing screens have been delivered. Work that lies ahead concentrates on hardening the matching engine, completing the payment and rating sub-systems, and conducting end-to-end verification under conditions resembling production.

7

CHAPTER 1 — INTRODUCTION

1.1 Background of the Study

The majority of large Indian employers run campus-style offices that pull thousands of employees from a wide catchment area on every working day. A fair share of these commuters share overlapping corridors yet still continue to drive their own vehicles. The downstream effects are widely understood: rush-hour congestion at gate entries, mounting strain on already-limited parking, larger fuel bills per employee, and a measurable contribution to vehicular emissions in the surrounding locality.

A number of open marketplaces today offer ride-hailing and even occasional ride-pooling. These services, however, treat every commuter as an anonymous customer, and that creates two issues for corporate use. First, an employer cannot reliably impose its safety expectations on a public platform. Second, the experience is purely transactional rather than community-driven, so the natural trust that already exists between colleagues from the same organization is left untapped.

1.2 Project Introduction

The Corporate Carpooling System proposed in this dissertation is a closed platform that admits only verified employees of a registered organization. Each user signs up with a corporate email address that is checked against the organization's domain, and a one-time password is despatched to confirm ownership of the inbox. Once verified, the user can act as a driver, a passenger, or both, depending on the day and the route. Drivers register their vehicle alongside its seating capacity and publish ride schedules with an intended origin, destination, and waypoint list. Passengers raise ride requests with their pickup and drop coordinates and the time window in which they would like to travel.

The matching engine sits on top of PostGIS. Driver routes are persisted as GEOMETRY (LineString, SRID 4326), passenger pickup and drop points are stored as GEOGRAPHY (Point, SRID 4326), and the engine uses ST_DWithin and ST_Distance to identify feasible matches without sweeping the entire ride table. Once both parties confirm, the platform takes over the operational layer and provides live tracking, in-ride chat, payment, ratings, an SOS safety channel, and the backup-driver path.

11

1.3 Goals and Objectives

The principal objectives of the project are listed below:

- Build a closed, employee-verified carpooling platform restricted to a single organization domain.
- Provide a geo-aware matching engine that ranks driver schedules by proximity, route overlap, and seat availability.
- Stream real-time vehicle location to matched passengers using Web Sockets, while persisting the trail in a ride_location_pings table for audit purposes.
- Offer an in-ride chat channel between drivers and passengers without exposing personal phone numbers.

- Implement an SOS workflow that alerts guardian contacts and shares the live ride location with them.
- Implement a backup-driver mechanism that automatically promotes a pre-ranked alternate driver if the primary cancels or stops responding.
- Capture payment intent and exchange ratings after every completed ride.
- Provide an admin dashboard that lets the organization administrator monitor active rides, review incidents, and manage users.

9

CHAPTER 2 — SYSTEM ANALYSIS

2.1 Existing System

At present, an employee who would like to share a ride to work has three loosely connected choices. The first is a paper- or chat-based arrangement with a colleague, in which schedules are coordinated by hand and forgotten the moment one party falls ill. The second is a public ride-hailing app that pools strangers together at a per-trip fare. The third is the company shuttle, which typically follows fixed routes at fixed times and does not bend around individual schedules.

2.2 Drawbacks of the Existing System

- Coordinating over chat does not scale; a driver running a daily pool ends up tracking seat counts in a paper notebook.
- Public ride-hailing pools mix unverified strangers, which is not acceptable for some categories of corporate users — especially female employees travelling outside peak hours.
- There is no shared internal route catalogue, so two employees living in the same colony often discover each other only by chance.
- Live tracking on public apps is tied to a paid trip; for an informal carpool, the only available method is a manual WhatsApp pin every few minutes.
- There is no audit trail. If a safety incident were to occur, there is no central record of who was on the ride, the path taken, or the timing.
- Backup-driver coverage simply does not exist. If the primary driver pulls out at the last moment, the passengers are left stranded.

2.3 Proposed System

The proposed Corporate Carpooling System unifies the three existing options into a single, organization-bounded platform. Verified employees register a single profile, declare the routes they typically commute on, and toggle between the driver and passenger roles on demand. The platform itself takes care of match discovery, accepting and rejecting requests, streaming live location to matched passengers, hosting a ride-scoped chat channel, and logging the entire ride lifecycle in a dedicated audit table.

Two safety primitives sit on top of the lifecycle. The SOS path lets any participant raise an alert that pushes a notification — together with the live location — to a list of guardian contacts that the user has registered in advance. The backup-driver path lets a passenger, or the system itself, promote a pre-ranked secondary driver into the primary slot whenever the original driver becomes unresponsive.

2.4 Scope of the Dissertation

The scope of this dissertation extends across the analysis, design, and partial implementation of the platform. The mid-semester deliverable is centred on the database schema, the back-end service tier, and the React front-end skeleton. The end-semester deliverable will widen the work to include a

hardened matching engine, the complete payment and rating flow, and an end-to-end test pass over the full ride lifecycle. Native mobile applications fall outside the scope of this dissertation; the same back-end, however, is being designed in a way that will accommodate a mobile client in a follow-on phase.

10

CHAPTER 3 — REQUIREMENTS SPECIFICATION

3.1 Functional Requirements

ID	Requirement
FR-01	Permit a new user to register with a corporate email address and verify it through an OTP.
FR-02	Authenticate registered users using a JWT-based login flow.
FR-03	Allow a user to maintain a profile that captures role (driver, passenger, or both), gender preference for matching, vehicle details (when a driver), and guardian contacts.
FR-04	Allow a driver to publish a ride schedule with origin, destination, waypoints, planned timings, seat capacity, and recurrence.
FR-05	Allow a passenger to post a ride request that carries pickup and drop coordinates and the desired time window.
FR-06	Run a geo-aware matching engine that ranks candidate driver schedules by proximity, route overlap, and seat availability.
FR-07	Allow the driver to accept or reject a passenger request and notify the passenger of the outcome.
FR-08	Stream the driver's live location to matched passengers and persist the location pings for audit.
FR-09	Provide a ride-scoped chat channel between drivers and accepted passengers.
FR-10	Allow any participant to raise an SOS that alerts the user's guardian contacts and shares the live ride location with them.
FR-11	Maintain a ranked list of backup drivers for every active ride and activate the next-in-line if the primary driver cancels or stops responding.
FR-12	Capture payment intent at the end of a ride and record the resulting transaction state.
FR-13	Allow the driver and passenger to rate each other once the ride is complete.
FR-14	Provide an admin view that lists active rides, recent SOS incidents, and user accounts for the organization administrator.

3.2 Non-Functional Requirements

Quality Attribute	Description
Security	Every API call apart from authentication requires a valid JWT. Passwords are stored using BCrypt. Sensitive endpoints are gated by role-based access.
Performance	Matching queries against a fifty-thousand-route dataset must return within five hundred milliseconds when the appropriate spatial indexes are in place.
Reliability	Each ride state transition is recorded in a ride_events table so that any mid-ride failure can be reconstructed afterwards.
Scalability	The service tier is stateless. Horizontal scaling is achieved by adding application instances behind a load balancer.
Usability	The front-end is a responsive single-page application. Users should be able to complete the request-and-match flow in fewer than five interactions.
Auditability	Lifecycle events, location pings, and SOS incidents are written to dedicated tables and are not removed once a ride completes.

3.3 Hardware and Software Requirements

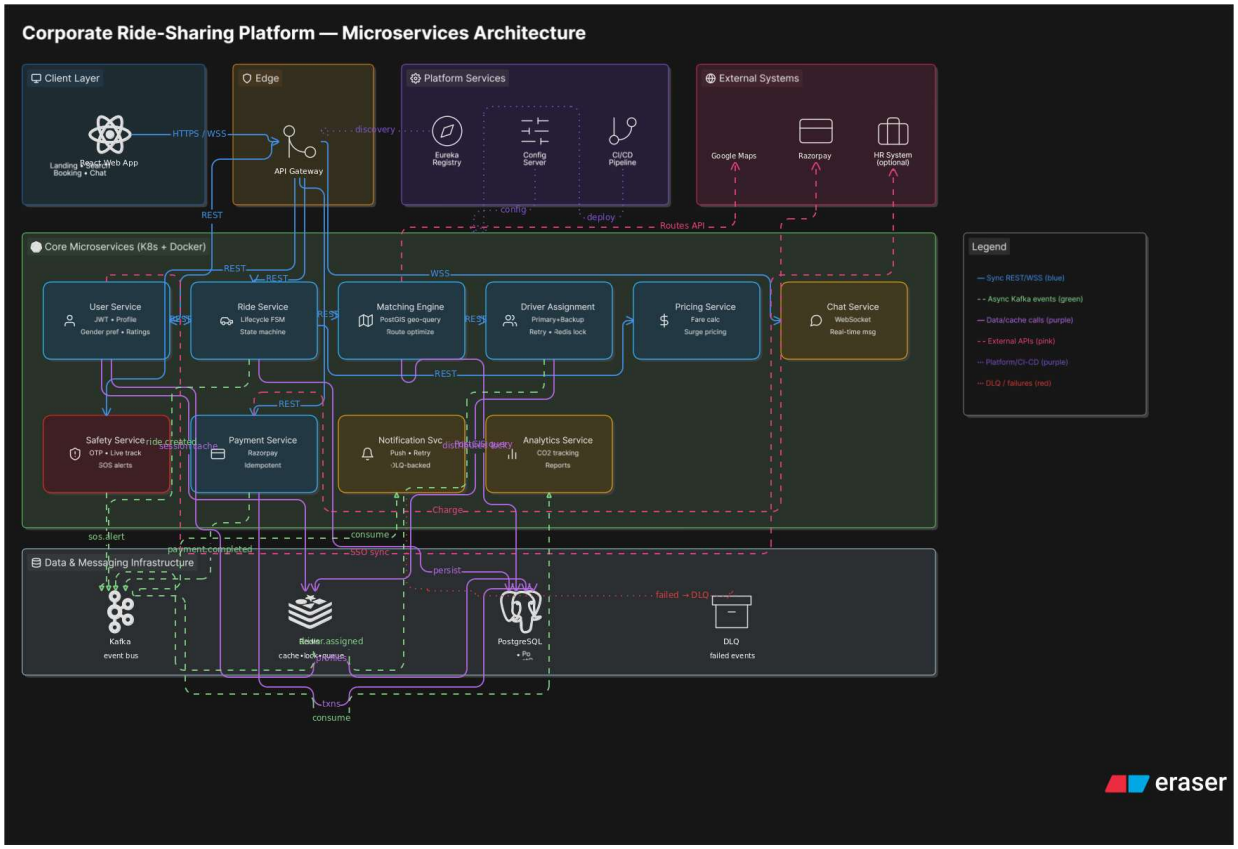
The development environment used throughout this project is summarised below.

Component	Specification
Workstation	Intel i5 or higher, 8 GB RAM minimum (16 GB recommended), 256 GB SSD.
Operating System	Windows 10/11, macOS 13+, or Ubuntu 22.04 LTS for development.
Backend Runtime	Java 18 LTS, Apache Maven 3.9, Spring Boot 3.3.
Frontend Runtime	Node.js 20 LTS, npm 10, React 18 with Vite 5.
Database	PostgreSQL 16 with the PostGIS 3 extension enabled.
Build & Source Control	Git 2.40+, GitHub for repository hosting.
IDE & Tooling	IntelliJ IDEA / VS Code, Postman for API testing, pgAdmin 4 for database administration.

CHAPTER 4 — SYSTEM DESIGN

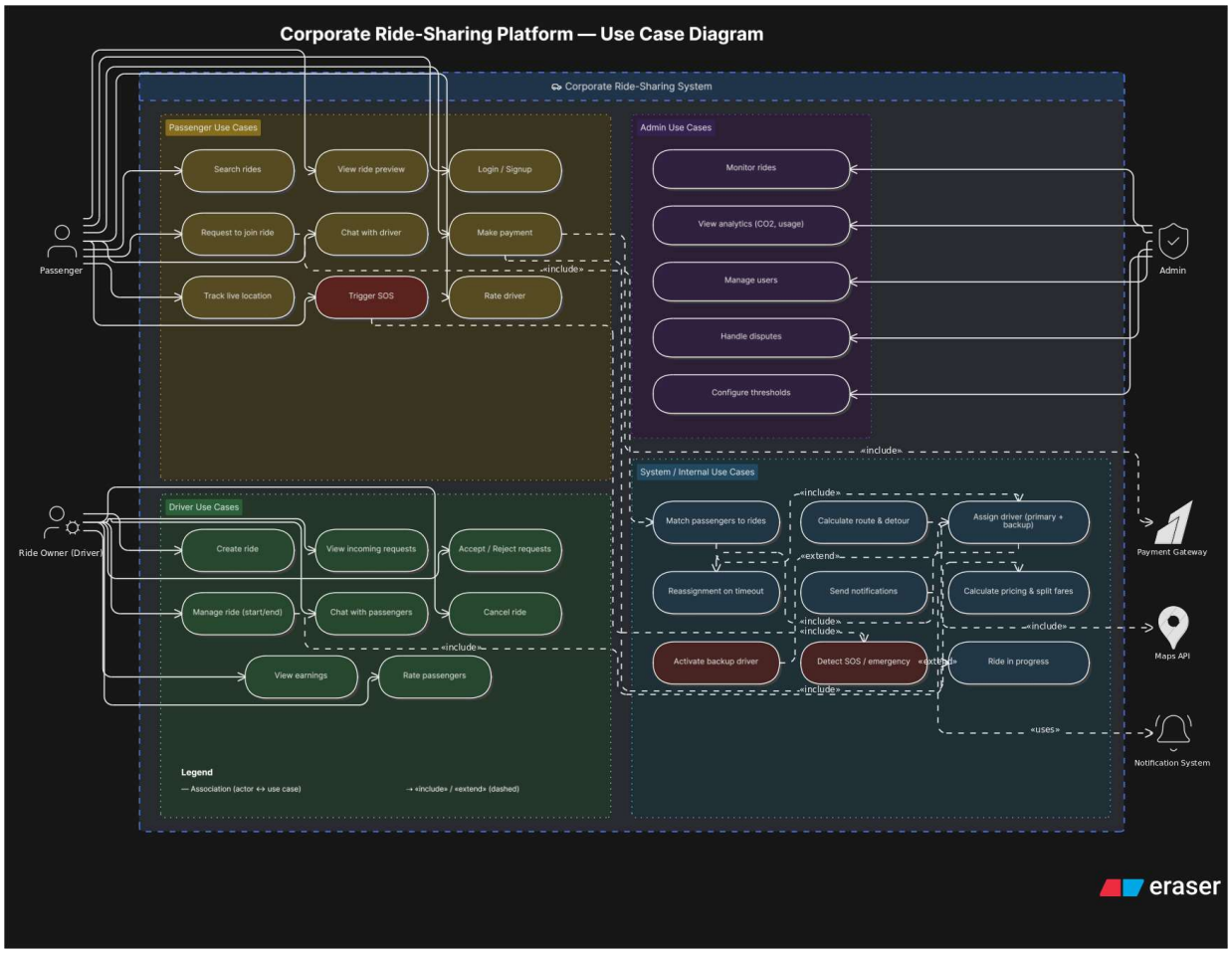
4.1 High-Level Architecture

The platform follows a three-tier architecture in which a React single-page front-end communicates with a stateless Spring Boot service tier, which in turn talks to a PostgreSQL database with PostGIS enabled. Web Sockets carry live tracking and chat traffic, while standard REST endpoints handle the remainder of the domain operations.



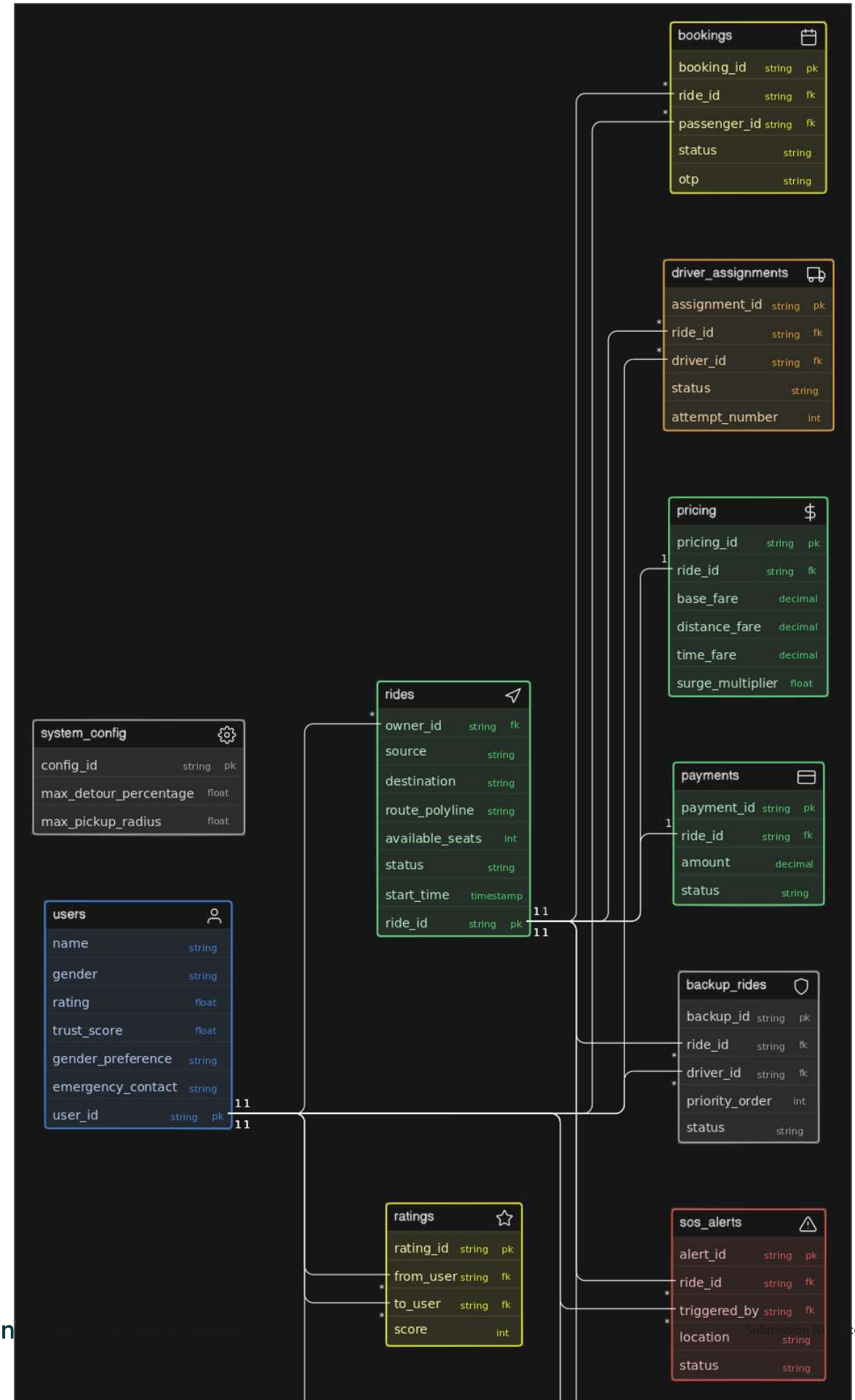
4.2 Use Case Diagram

The use case diagram below captures the principal interactions between the three actor classes — driver, passenger, and administrator — and the system.



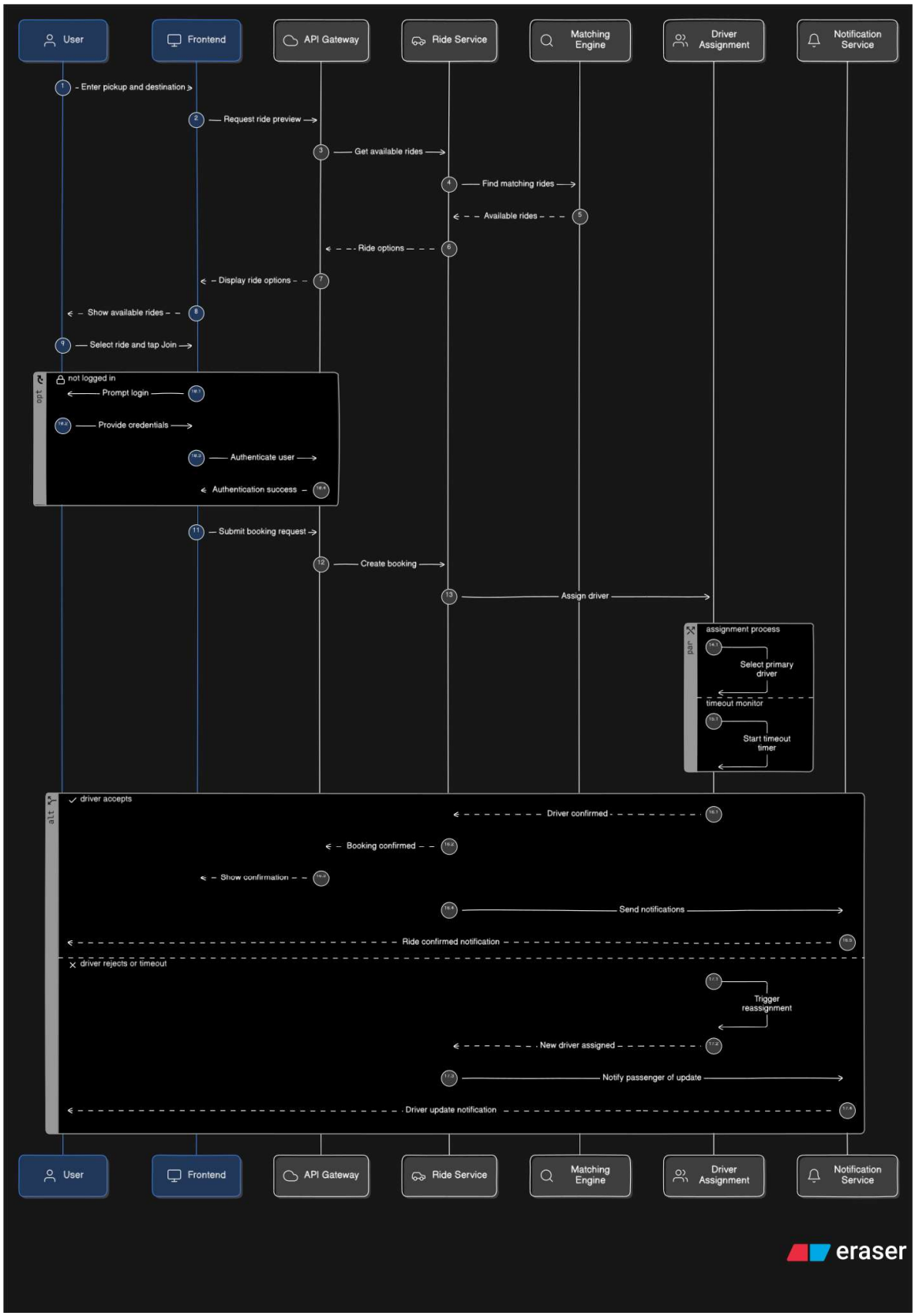
4.3 Entity Relationship Diagram

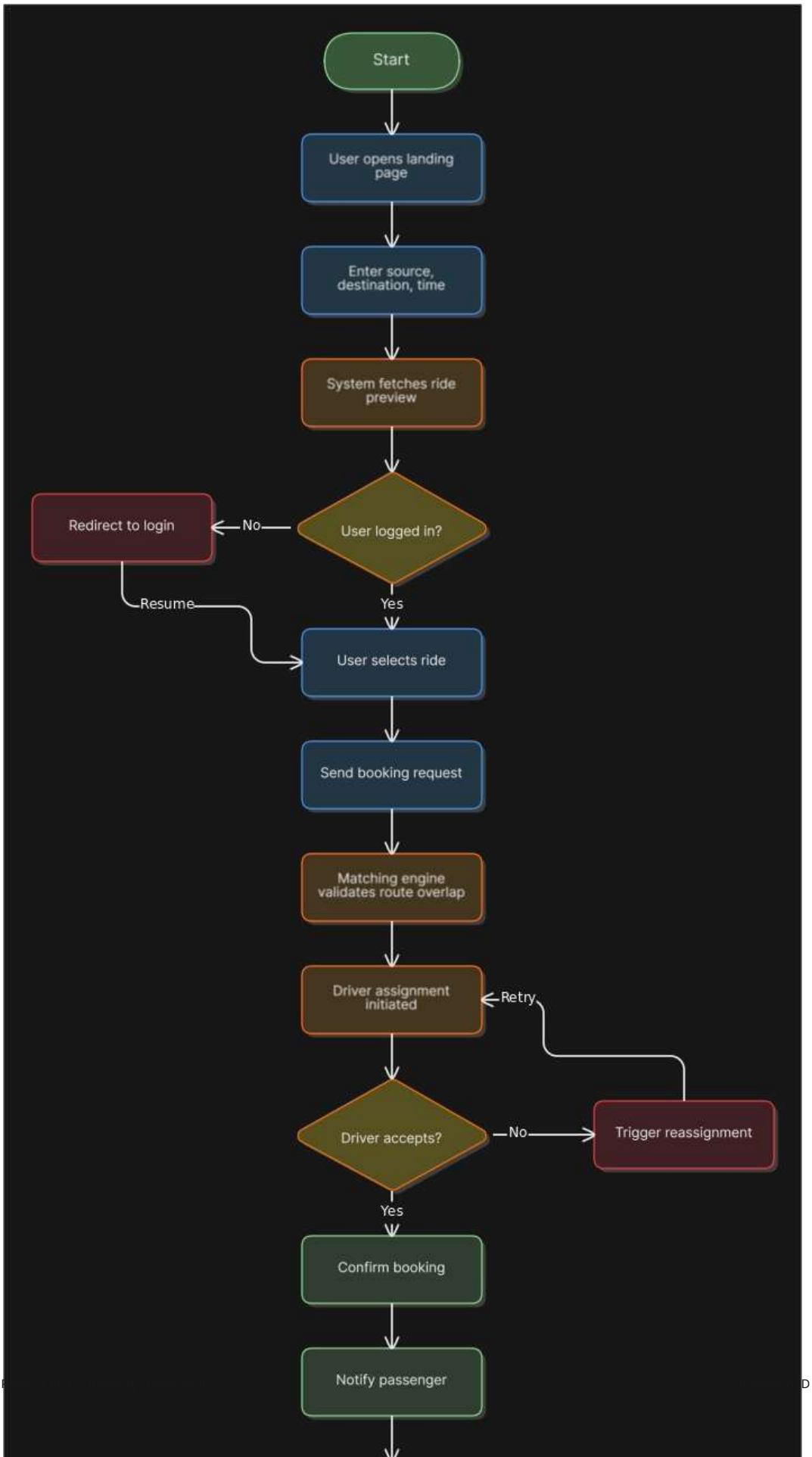
The entity relationship diagram shows how the sixteen domain entities relate to one another, with particular emphasis on the spatial columns that drive the matching engine.


bookings
booking_id string pk
ride_id string fk
passenger_id string fk
status string
otp string
driver_assignments
assignment_id string pk
ride_id string fk
driver_id string fk
status string
attempt_number int
pricing
pricing_id string pk
ride_id string fk
base_fare decimal
distance_fare decimal
time_fare decimal
surge_multiplier float
rides
owner_id string fk
source string
destination string
route_polyline string
available_seats int
status string
start_time timestamp
ride_id string pk
system_config
config_id string pk
max_detour_percentage float
max_pickup_radius float
payments
payment_id string pk
ride_id string fk
amount decimal
status string
users
name string
gender string
rating float
trust_score float
gender_preference string
emergency_contact string
user_id string pk
backup_rides
backup_id string pk
ride_id string fk
driver_id string fk
priority_order int
status string
ratings
rating_id string pk
from_user string fk
to_user string fk
score int
sos_alerts
alert_id string pk
ride_id string fk
triggered_by string fk
location string
status string

4.4 Sequence Diagram

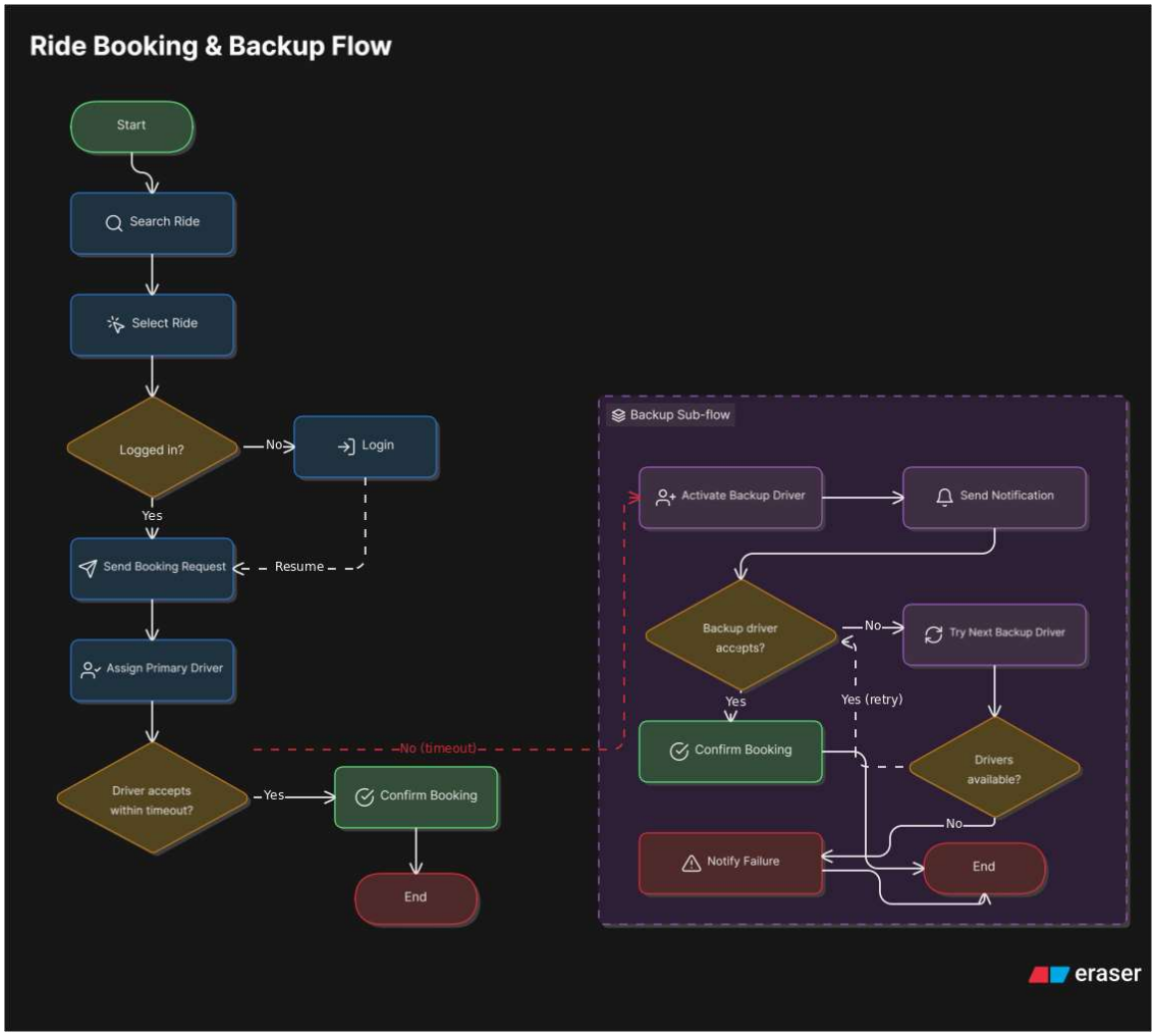
The sequence diagrams trace the message exchange across the front-end, back-end, and database for the registration-and-match flow as well as the live tracking pipeline.

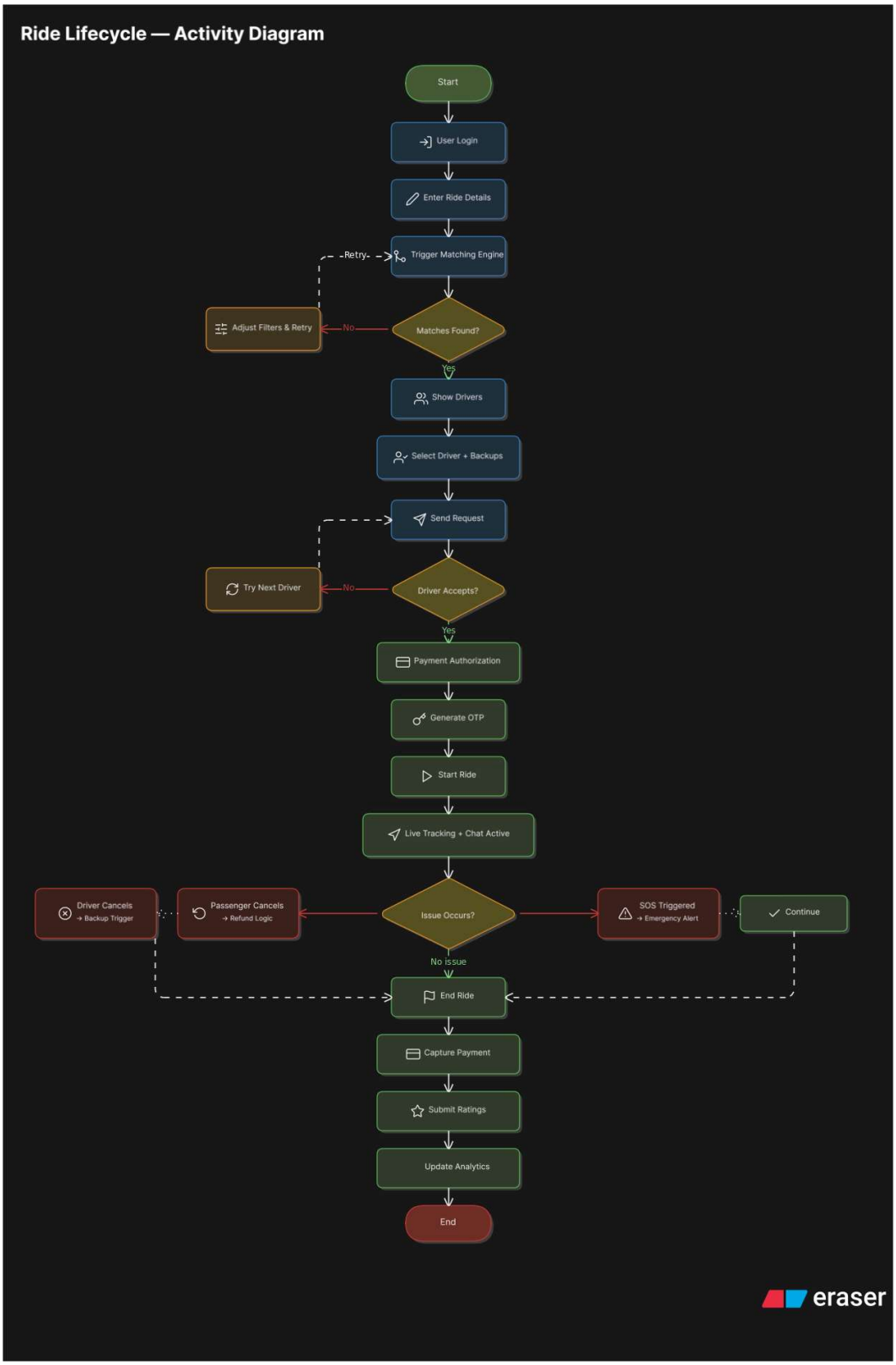

User
Frontend
API Gateway
Ride Service
Matching Engine
Driver Assignment
Notification Service
1 Enter pickup and destination
2 Request ride preview
3 Get available rides
4 Find matching rides
5 Available rides
6 Ride options
7 Display ride options
8 Show available rides
9 Select ride and tap Join
opt
not logged in
10.1 Prompt login
10.2 Provide credentials
10.3 Authenticate user
10.4 Authentication success
11 Submit booking request
12 Create booking
13 Assign driver
par
assignment process
14.1 Select primary driver
timeout monitor
15.1 Start timeout timer
alt
driver accepts
16.1 Driver confirmed
16.2 Booking confirmed
16.3 Show confirmation
16.4 Send notifications
16.5 Ride confirmed notification
driver rejects or timeout
17.1 Trigger reassignment
17.2 New driver assigned
17.3 Notify passenger of update
17.4 Driver update notification
eraser


Start
User opens landing page
Enter source, destination, time
System fetches ride preview
User logged in?
No
Redirect to login
Resume
Yes
User selects ride
Send booking request
Matching engine validates route overlap
Driver assignment initiated
Retry
Driver accepts?
No
Trigger reassignment
Yes
Confirm booking
Notify passenger

4.5 Activity Diagrams

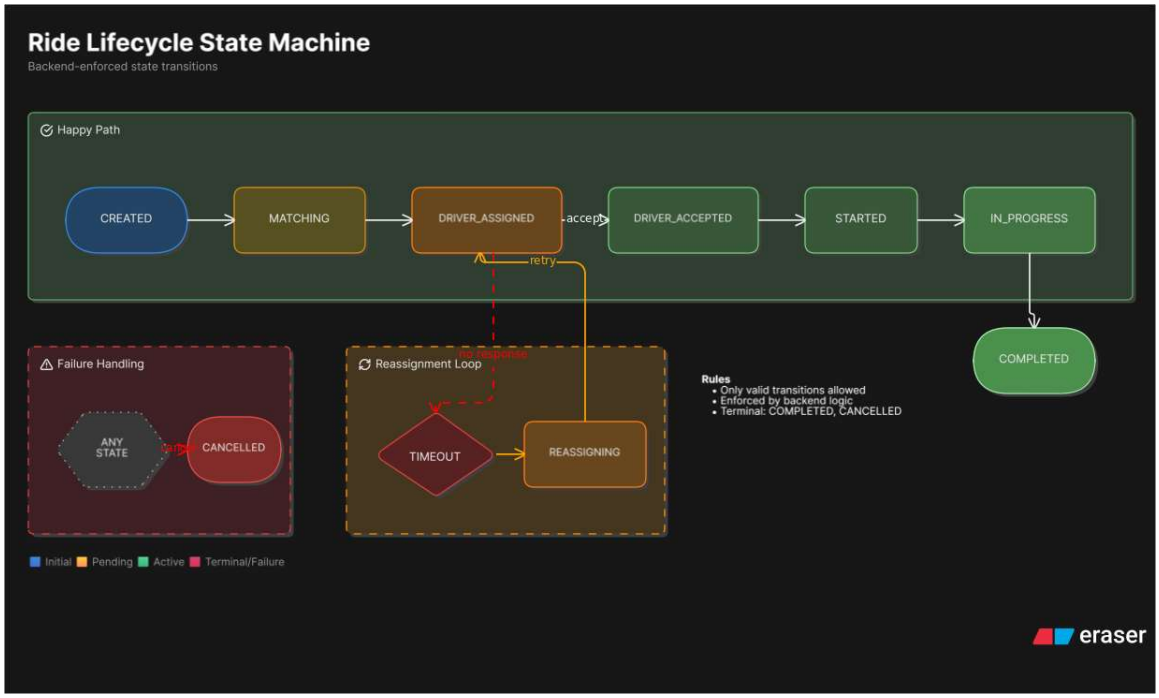
The activity diagrams illustrate the higher-level workflow for ride creation and the decision points that determine whether a passenger request is matched, queued, or rejected.





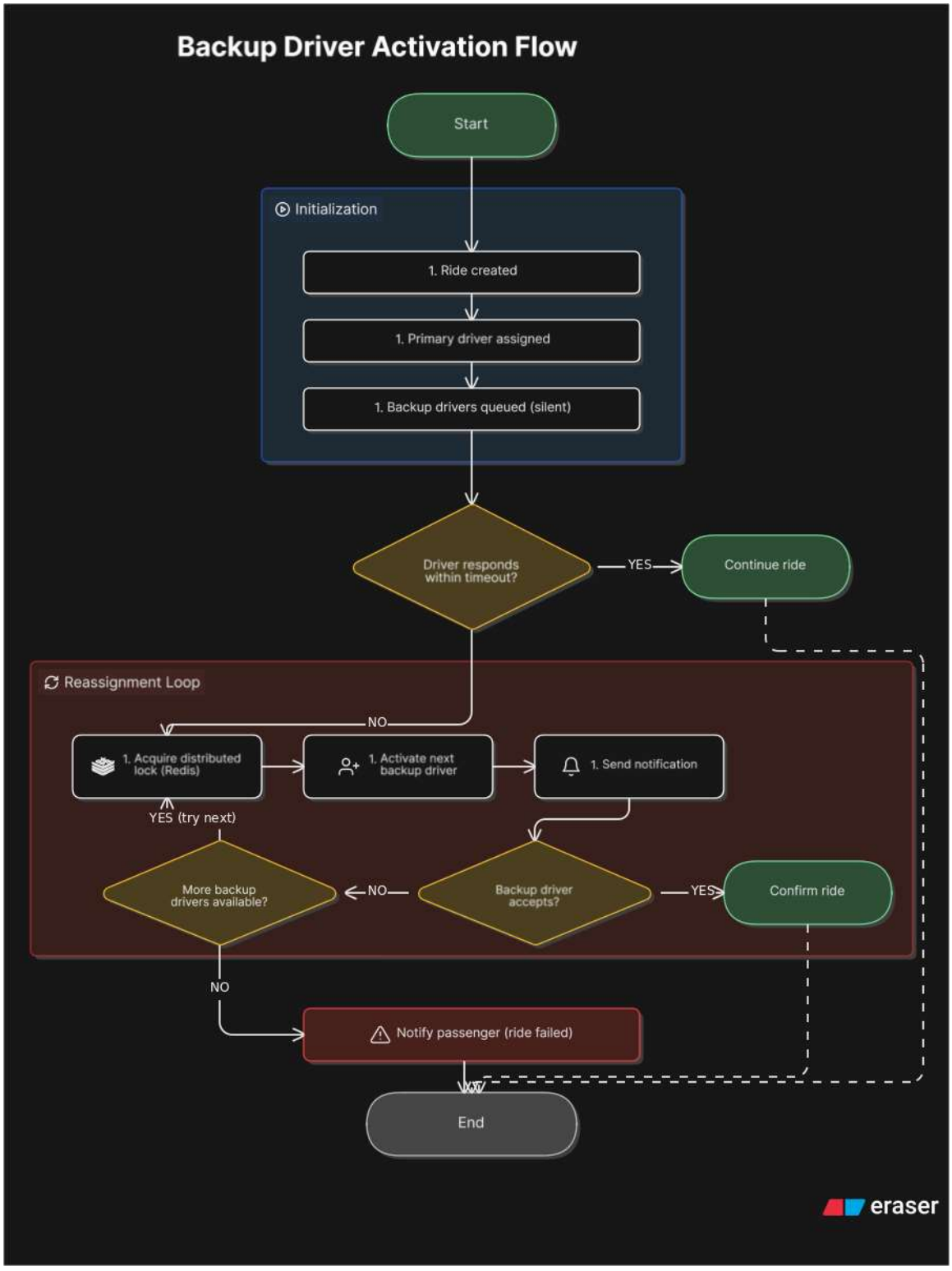
4.6 Ride Lifecycle State Machine

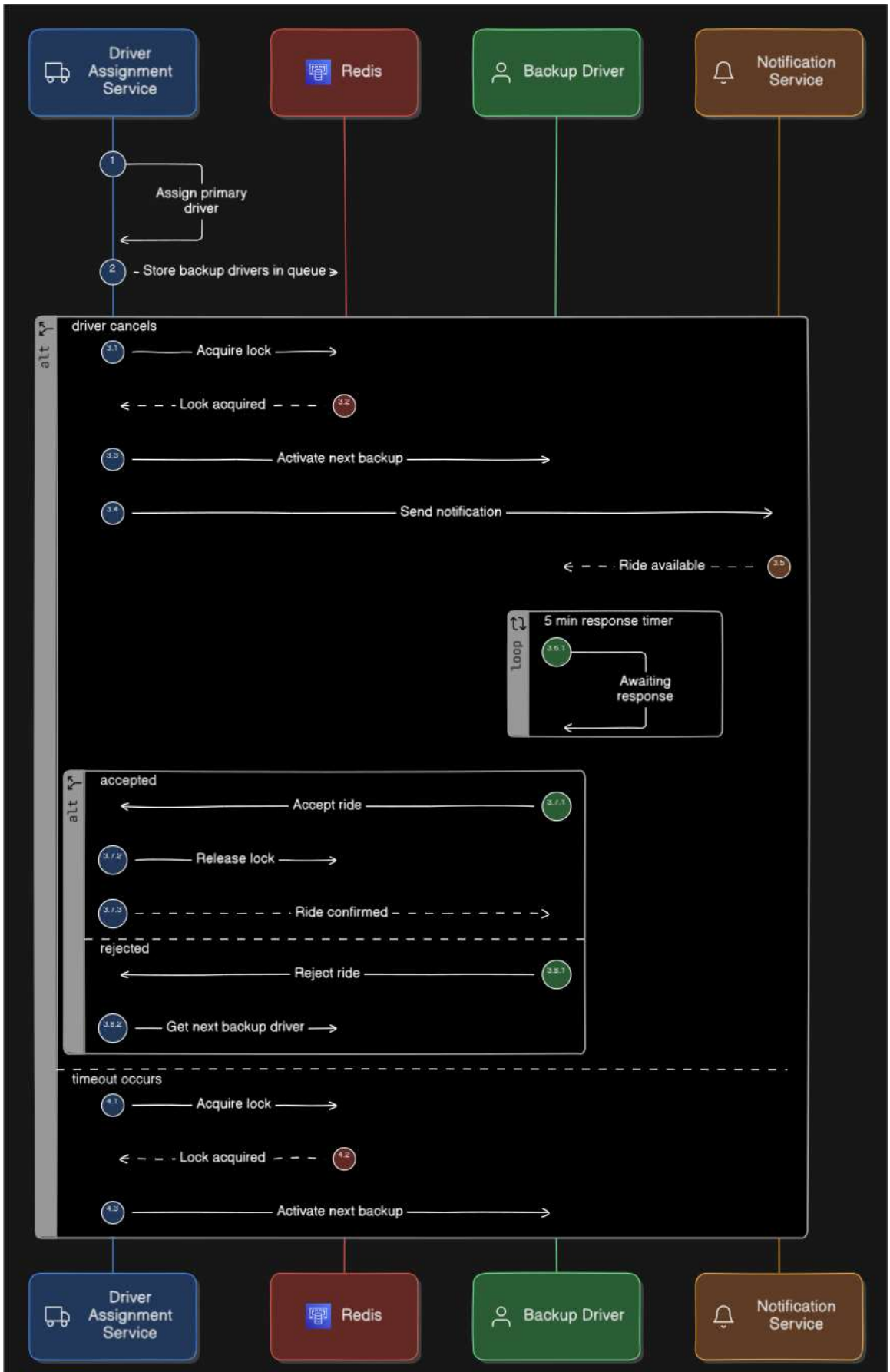
The ride lifecycle is modelled as an explicit state machine to ensure that every transition is auditable and that invalid transitions are rejected at the service layer.



4.7 Backup Driver Activation Flow

The diagrams below describe the conditions that trigger backup-driver activation, the order in which alternates are considered, and the notifications that are dispatched to the affected riders.


Backup Driver Activation Flow
Start
Initialization
1. Ride created
1. Primary driver assigned
1. Backup drivers queued (silent)
Driver responds within timeout?
YES
Continue ride
Reassignment Loop
NO
1. Acquire distributed lock (Redis)
1. Activate next backup driver
1. Send notification
YES (try next)
More backup drivers available?
NO
Backup driver accepts?
YES
Confirm ride
NO
Notify passenger (ride failed)
End
eraser


Driver Assignment Service
Redis
Backup Driver
Notification Service
1
Assign primary driver
2
Store backup drivers in queue
alt
driver cancels
3.1
Acquire lock
Lock acquired
3.2
3.3
Activate next backup
3.4
Send notification
Ride available
3.5
loop
5 min response timer
3.6.1
Awaiting response
alt
accepted
Accept ride
3.7.1
3.7.2
Release lock
3.7.3
Ride confirmed
rejected
Reject ride
3.8.1
3.8.2
Get next backup driver
timeout occurs
4.1
Acquire lock
Lock acquired
4.2
4.3
Activate next backup
Driver Assignment Service
Redis
Backup Driver
Notification Service
eraser

CHAPTER 5 — IMPLEMENTATION PROGRESS

The current state of the implementation is captured in the table below.

Module / Deliverable	Status
Database schema (PostGIS)	Completed
Email-verification migration	Completed
Spring Boot project bootstrap	Completed
Sixteen JPA entities	Completed
Sixteen REST controllers	Completed
Authentication & JWT	Completed
WebSocket tracking & chat	Completed
Geo-aware matching engine (v1)	Completed
Backup-driver service	Completed
React front-end skeleton (10 screens)	Completed
Admin dashboard	Completed
Payment & rating sub-system	In progress
End-to-end test suite	Pending
Performance tuning of the matching engine	Pending
Production deployment runbook	Pending
Google Maps Platform — Maps JavaScript API	In progress
Google Maps Platform — Geocoding API	In progress
Google Maps Platform — Directions API	Pending
Google Maps Platform — Places Autocomplete	Pending
Route polyline overlay (React + @react-google-maps/api)	Pending

CHAPTER 6 — DETAILED PLAN OF WORK

#	Task	Duration	Specific Deliverable	Status
1	Requirements gathering and architecture finalisation	1 week	Architecture document; lists of functional and non-functional requirements	Completed
2	Database schema design with PostGIS	1 week	schema.sql containing sixteen tables, ENUM types, and spatial indexes	Completed
3	Spring Boot bootstrap, JPA entities, and authentication	2 weeks	Sixteen entities, JWT-based auth, OTP-based email verification	Completed
4	REST controllers, services, and WebSocket plumbing	2 weeks	Sixteen controllers, seventeen services, STOMP endpoints for tracking and chat	Completed
5	React front-end skeleton with design tokens and shared components	1 week	Tokens, WpButton family, ten screens wired to back-end APIs	Completed
6	Mid-semester report	1 week	This report along with its supporting diagrams	Completed
7	Development Phase 1 — Google Maps integration & matching hardening	18 Apr – 7 May 2026	Maps JS API, Geocoding API, Directions API, Places Autocomplete, route overlay in React, refined PostGIS matching queries	In progress
8	Development Phase 2 — Payment, ratings, and AI agent completion	8 May – 27 May 2026	Razorpay integration, ratings flow, refund handling, AI agent module finalisation, additional React screens	Upcoming
9	Testing & Optimisation	28 May – 12 Jun 2026	Spatial query benchmark, Cypress E2E suite, load-test report, bug-fix pass	Upcoming
10	Final Report & Evaluation	13 Jun – 25 Jun 2026	Final dissertation report, supervisor sign-off, viva-ready demonstration, deployment runbook	Upcoming
11	Final Submission & Closure	25 Jun – 11 Jul 2026	BITS portal submission, viva presentation, source-code archive, project closure	Upcoming

CHAPTER 7 — CONCLUSION OF MID-SEMESTER PHASE

By the close of the mid-semester phase, the dissertation has yielded a working vertical slice of the platform. The PostGIS-backed database schema, the Spring Boot service tier with sixteen entities and sixteen controllers, the React front-end with ten screens plus an admin dashboard, and the supporting design diagrams together demonstrate that the architecture holds up under implementation. The geo-aware matching path, along with the safety-critical paths for SOS and backup-driver activation, has been built and is reachable from the user interface.

The remaining effort focuses on three areas. First, the payment and rating sub-system needs to be integrated with a sandbox payment provider so that the post-completion flow becomes production-grade. Second, the matching query needs benchmark-driven tuning to ensure that it stays inside the five-hundred-millisecond budget on a realistic dataset. Third, an end-to-end test suite must exercise the full ride lifecycle so that regressions are caught before deployment. These items are scoped in the plan of work above and are expected to land comfortably within the post-mid-semester window.

REFERENCES

1. Spring Boot Reference Documentation, Pivotal Software / VMware. https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/

2. Spring Security Reference, VMware. https://docs.spring.io/spring-security/reference/

3. PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/

4. PostGIS 3 Documentation. https://postgis.net/documentation/

5. Hibernate Spatial Documentation. https://hibernate.org/orm/spatial/

6. React Documentation. https://react.dev/

7. Vite Build Tool Documentation. https://vite.dev/

8. JJWT — Java JSON Web Token Library. https://github.com/jwtk/jjwt

9. STOMP Over WebSocket, Spring Framework. https://docs.spring.io/spring-framework/reference/web/websocket/stomp.html

10. Agatz, N., Erera, A., Savelsbergh, M., and Wang, X. Optimization for Dynamic Ride-Sharing: A Review. European Journal of Operational Research, 2012.

11. Furuhata, M., Dessouky, M., Ordonez, F., Brunet, M.-E., Wang, X., and Koenig, S. Ridesharing: State-of-the-Art and Future Directions. Transportation Research Part B, 2013.

12. Razorpay Standard Checkout Integration Guide. https://razorpay.com/docs/payments/payment-gateway/web-integration/standard/